v1.0.0
20260215

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[Lab Notes]

MULTITHREADED PROGRAMMING IN JAVA: INTRODUCTION

MULTITHREADED PROGRAMMING IN JAVA

- Java has been the first mainstream programming language to provide a first native support to concurrent programming
  - “conservative approach”: everything is still an object
  - + mechanisms for concurrency
- Extended with the **java.util.concurrent library** (JSR 166 and JSR 236) to provide a higher level support to concurrent programming
  - semaphores, locks, synchronizers, etc
  - task frameworks (discussed in other Lab Notes)
- **Virtual threads** introduced with JDK 21, proposed with JEP (JDK Enhancement Proposal) 444
  - introducing lightweight user-level threads in Java
  - Discussed in other Lab Notes

BASIC MECHANISMS: OVERVIEW

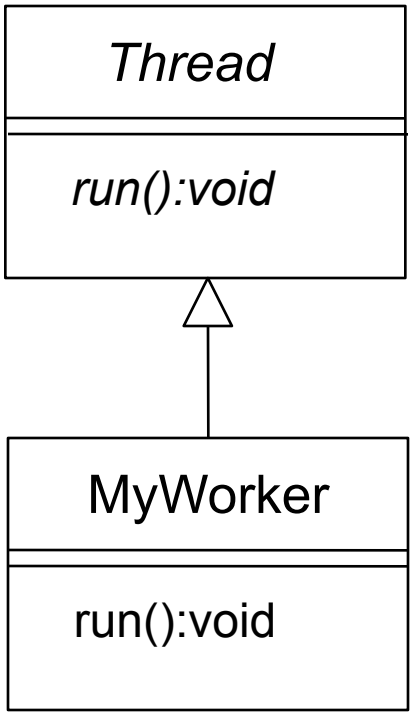
- Class **`Thread`** (along with a few related utility class) used to initiate and control concurrent activities
  - `Runnable` interface
- Keyword **`synchronized`** and **`volatile`**, used to control the execution of code in objects that may participate in multiple threads
  - for mutual exclusion
- Methods **`wait`**, **`notify`**, and **`notifyAll`** as defined in `java.lang.Object` used to coordinate activities across threads
  - for real synchronization
- In this module we focus on thread creation and running

DEFINING THREADS

- Java provides a basic API for defining new types of thread, and for dynamically create and (partially) manage thread execution
  - threads are mapped onto OS threads, with strategies that depend on the specific system
  - typically a one-to-one approach is adopted
- A thread is represented by the abstract class **`Thread`**, characterised by the abstract method **`run`**, which defines the behaviour of the thread
  - a concrete thread can be defined by extending `Thread` class, and implementing the run method
- To start thread asynchronous execution, the method **`start`** is provided
  - must be invoked on the instance of a thread object
  - it returns immediately, and a new activity executing what specified in **`run`** method is launched
- The thread terminates as soon as the execution of the method run body is completed

`Thread` CLASS

- `Thread` class is provided in the package `java.lang`




```
public class MyWorker extends Thread  {

  public MyWorker(String name){
    super(name);
  }

  public void run(){
    ...
    <active behaviour>
    ...
  }
}
```

MAIN THREAD API

- Main features provided by the `Thread` class:
  - `Thread(String name)`
    - to construct a thread with a specified name
  - String **getName**();
    - get the thread name
  - `void` **`sleep`**`(long ms)`
    - to suspend thread execution for ms milliseconds
  - void **`join`**`()`
    - wait for the termination of the thread
  - `void` **`interrupt`**`()`
    - causes a sleep, wait or join to abort with an InterruptedException, which can be caught and deal with in an application-specific way
  - `static Thread` **`currentThread`**`()`
    - to get the reference to current thread in execution

DEPRECATED

- All the public methods to asynchronously act on the control flow of the thread have been deprecated
  - `stop`
  - `suspend`
  - `resume`
  - `destroy`
  - ...
- The same functionality is achieved through proper patterns
  - next lab modules

SPAWNING THREADS

```
public class Test {
  public static void main(String[] args) {
     Thread myWorkerA = new MyWorker("worker-A");
     myWorkerA.start();
     Thread myWorkerB = new MyWorker("worker-B");
     myWorkerB.start();
  }
}
```

- **NOTE**
  - the method executed on the thread object is **`start`**, not **`run`**
    - *what if we execute the method `run` instead? what is the behaviour of the program Test if we invoke `run` instead of `start` for both the workers?*
  - a Java application has always at least one thread in execution

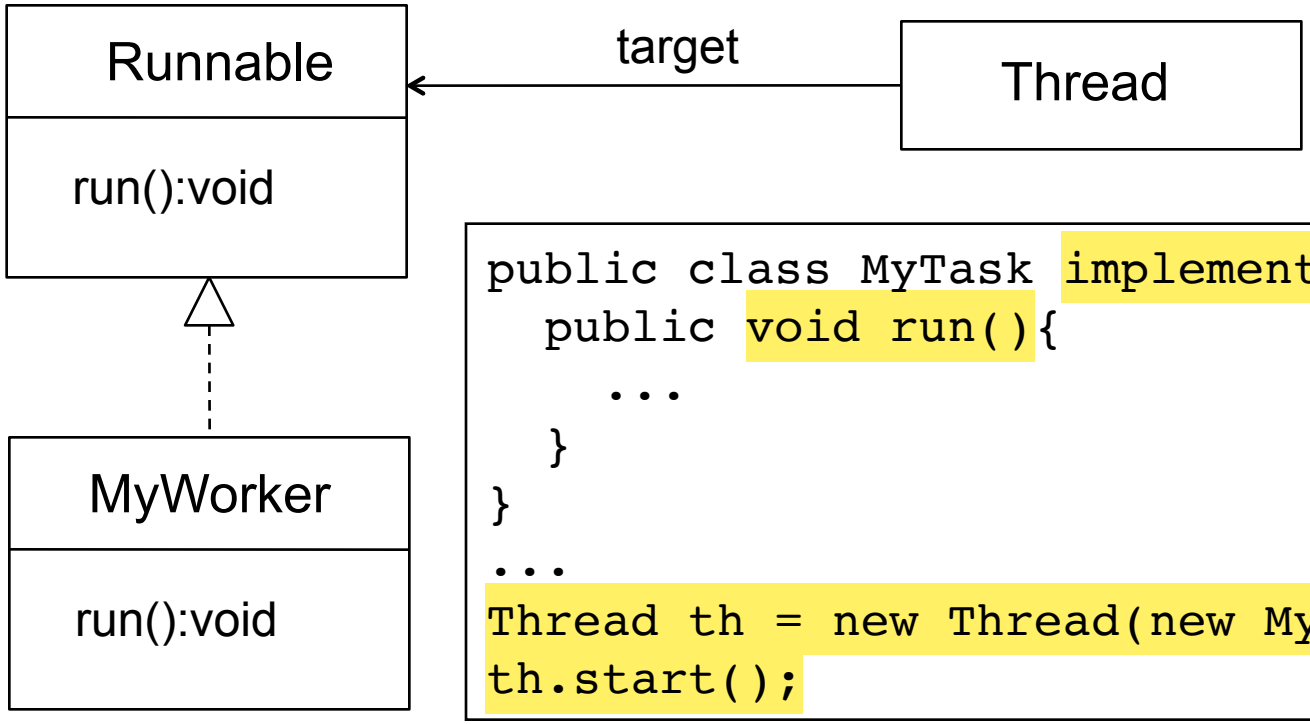
JOINING THREADS

- The join method allows for a thread to synchronize its execution with the termination of an another thread
  - in particular: t.join() suspends the current thread until the thread t has completed its execution
- Example:

```
MyThread t = new MyThread();
t.start();
…
t.join();
System.out.println("spawned thread terminated.");
```

`Runnable` Interface

- An alternative approach is provided to define a thread, based on `Runnable` interface, useful when the class used to implement the thread belongs to some class hierarchy (utile usare l'interfaccia perchè in Java abbiamo ereditarietà singola, non multipla)
  - ..already extending some class, which is not `Thread`




```
public class MyTask implements Runnable {
  public void run(){
    ...
  }
}
...
Thread th = new Thread(new MyTask());
th.start();
```

- Note the `Runnable` object parameter in `Thread` constructor

MONITORING THREADS: JConsole TOOL

- JConsole is the Java Monitoring and Management Console, a graphical tool shipped in J2SE JDK 5.0 (and later versions)
  - it uses the instrumentation of the Java virtual machine to provide information on performance and resource consumption of applications running on the Java platform
  - based on the Java Management Extension (JMX) technology
  - http://java.sun.com/developer/technicalArticles/J2SE/jconsole.html
- Useful (also) to monitor the thread spawned by a running Java programs
  - including VM threads, such as the one used for garbage collecting

A OPEN-SOURCE PROFILER: VISUALVM

- Similar to JConsole, VisualVm is a full-fledged **profiler** that allow for measuring and visualizing performances of Java programs
  - like JConsole, it uses the instrumentation of the Java virtual machine to provide information on performance and resource consumption of applications running on the Java platform
    - based on the Java Management Extension (JMX) technology
  - shipped with J2SE JDK 5.0 (and later versions)
  - http://visualvm.java.net
- More fine-grained monitoring than JConsole
  - monitoring % CPU used by methods, threads
  - monitoring how long a thread is blocked or running
  - ...

FURTHER TOOLS: FLIGHT REC & JMC

- Toolchain to continuously collect low level and detailed runtime information enabling after-the-fact incident analysis
  - **Java Flight Recorder**
    - a profiling and event collection framework built into the Oracle JDK.
    - It allows Java administrators and developers to gather detailed low level information about how the Java Virtual Machine (JVM) and the Java application are behaving
  - **JDK Mission Control**
    - an advanced set of tools that enables efficient and detailed analysis of the extensive of data collected by Java Flight Recorder
    - enables developers and administrators to collect and analyze data from Java applications running locally or deployed in production environments
- https://www.oracle.com/java/technologies/jdk-mission-control.html

A PROGRAMMING DISCIPLINE

- Viewing threads as active objects **encapsulating** state, behaviour and **the control of the behaviour**
  - the object's methods should be called only by the thread represented by the object
    - the use of public methods should be minimized
- Promoting interaction by means of shared (passive) objects
  - not by calling public methods of their interface
    - this would violate encapsulation of control
- Strong conceptual separation between *active* and *passive* entities
  - active entities as agents that are responsible of accomplishing some tasks
  - passive entities as the objects shared and manipulated by such agents in order to accomplish such tasks